

Investigating SQL Join Commands — Query Review

Query Review 1 — Employee + Department Name

SQL:

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
JOIN departments d
  ON e.employee_id = d.department_id;
```

Sample Data:

employees

employee_id	first_name	last_name	department_id
1	Ali	Hassan	10
2	Sara	Khan	20
3	Omar	Saleh	10
4	Noor	Amin	30

departments

department_id	department_name
1	HR
2	Sales
3	IT
4	Finance

Logical Analysis:

WRONG LOGIC. The ON clause joins `e.employee_id = d.department_id`, matching an employee's ID number to a department's ID number. These are two completely unrelated IDs. The join appears to return results only because small employee IDs (1, 2, 3, 4) coincidentally match small department IDs (1, 2, 3, 4), but the data is semantically incorrect — Ali (`employee_id=1`) actually belongs to department 10 (IT), not HR. The correct join condition must use the actual foreign key column: `e.department_id = d.department_id`.

Query Review 2 — Employee + Job Title

SQL:

```

SELECT e.first_name, e.last_name, j.job_title
FROM employees e
JOIN jobs j
    ON e.employee_id = j.job_id;

```

Sample Data:

employees

employee_id	first_name	last_name	job_id
1	Ali	Hassan	J03
2	Sara	Khan	J01
3	Omar	Saleh	J02
4	Noor	Amin	J01

jobs

job_id	job_title
J01	Analyst
J02	Manager
J03	Developer

Logical Analysis:

WRONG LOGIC. The ON clause compares e.employee_id (an integer such as 1, 2, 3) with j.job_id (a string such as 'J01', 'J02'). Because the data types and formats are completely incompatible, no rows will ever match and the INNER JOIN returns zero rows every time. The employees table already contains a job_id foreign key column specifically designed for this relationship. The correct join condition is e.job_id = j.job_id.

Query Review 3 — Employees in Sales (LEFT JOIN)

SQL:

```

SELECT e.first_name, d.department_name
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id
WHERE d.department_name = 'Sales';

```

Sample Data:

employees

employee_id	first_name	department_id
1	Ali	10

2	Sara	20
3	Omar	20
4	Noor	NULL

departments

department_id	department_name
10	IT
20	Sales
30	Finance

Logical Analysis:

LOGIC ISSUE — the LEFT JOIN is pointless as written. The JOIN condition itself is correct (e.department_id = d.department_id), but the WHERE clause d.department_name = 'Sales' silently converts the LEFT JOIN into an effective INNER JOIN. For employees who have no department (like Noor, whose department_id is NULL), the LEFT JOIN produces NULL for d.department_name. Since NULL does not equal 'Sales', those employees are filtered out by the WHERE clause anyway. The final result is identical to what a plain INNER JOIN would return, making the LEFT JOIN meaningless. If the goal is to retrieve only Sales employees, replace LEFT JOIN with INNER JOIN. If the goal is to list all employees while showing the Sales label only where it applies, move the filter into the JOIN condition: ON e.department_id = d.department_id AND d.department_name = 'Sales'.

Query Review 4 — Employee + Their Manager (Self-Join)

SQL:

```
SELECT e.first_name, m.first_name AS manager_name
FROM employees e
JOIN employees m
ON e.employee_id = m.manager_id;
```

Sample Data:

employees (used as both e and m in the self-join)

employee_id	first_name	manager_id
1	Ali (CEO)	NULL
2	Sara	1
3	Omar	1
4	Noor	2

Logical Analysis:

WRONG LOGIC — the self-join condition is reversed. The ON clause `e.employee_id = m.manager_id` asks: 'find rows in m where m.manager_id equals e's own employee_id,' which effectively identifies employees who are themselves managers of others — not employees paired with their own manager. The query partially appears to work for Ali's direct reports (Sara and Omar) by coincidence, but Noor — who reports to Sara — is missing entirely from the results. The correct condition is `e.manager_id = m.employee_id`, which reads: 'match the employee's manager_id to the manager's employee_id,' properly pairing each employee with the person they report to.